Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LESSON PLAN

For

WSQ CompTIA Certified Cloud+ Training

TGS Ref No: TGS-2024049214

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 2.0

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	1 Jul 2026	First release. 36 hands-on labs mapped to CV0-004 domains 1 –6.	Dr. Alfred Ang
2.0	19 Aug 2026	Major revision. Deck expanded with full per-domain teaching content covering every CV0-004 exam sub-objective (31 blueprint topics, 500+ visual slides); labs restructured into one folder per lab; free web-tool kit (IP Calculator, PCAP Analyzer, Regex Generator, Cybersecurity Simulator) integrated into the labs.	Dr. Alfred Ang

TABLE OF CONTENTS

Course Overview

Learning Outcomes

Exam Domain Coverage

Daily Schedule

Day 1 — Cloud Architecture, Deployment & Operations

Day 2 — Security, DevOps, Troubleshooting & Assessment

Topic-by-Topic Breakdown

Domain 1 — Cloud Architecture (23%) — Deck: divider slide 19; Slides 135–142

Lab 1 — Cloud Service Models & Shared Responsibility · Slide 135

Lab 2 — High Availability with HAProxy & Keepalived · Slide 136

Lab 3 — Cloud Networking with VPC Namespaces · Slide 137

Lab 4 — Storage Tiers (Block, Object, File) · Slide 138

Lab 5 — Microservices & Service Discovery · Slide 139

Lab 6 — Containerization with Docker · Slide 140

Lab 7 — Virtualization with QEMU/KVM · Slide 141

Lab 8 — Relational vs Non-Relational Databases · Slide 142

Domain 2 — Deployment (19%) — Deck: divider slide 143; Slides 226–232

Lab 9 — Public, Private & Hybrid Cloud Models · Slide 226

Lab 10 — Blue-Green Deployment with Nginx · Slide 227

Lab 11 — Canary Deployment with HAProxy · Slide 228

Lab 12 — Rolling Deployment with Docker Compose · Slide 229

Lab 13 — Infrastructure as Code with Terraform · Slide 230

Lab 14 — Configuration as Code with Ansible · Slide 231

Lab 15 — JSON & YAML Scripting Logic · Slide 232

Domain 3 — Operations (17%) — Deck: divider slide 233; Slides 304–309

Lab 16 — Observability with Prometheus & Grafana · Slide 304

Lab 17 — Centralized Logging with the ELK Stack · Slide 305

Lab 18 — Distributed Tracing with Jaeger · Slide 306

Lab 19 — Horizontal Auto-Scaling Simulation · Slide 307

Lab 20 — Backup & Recovery with restic · Slide 308

Lab 21 — Patching & Lifecycle Management · Slide 309

Domain 4 — Security (19%) — Deck: divider slide 310; Slides 401–407

Lab 22 — Vulnerability Scanning with Trivy · Slide 401

Lab 23 — CIS Benchmark Audit with Lynis · Slide 402

Lab 24 — IAM & RBAC with Keycloak · Slide 403

Lab 25 — Bastion Host with SSH Key + MFA · Slide 404

Lab 26 — Secrets Management with HashiCorp Vault · Slide 405

Lab 27 — Web Application Firewall with ModSecurity · Slide 406

Lab 28 — Detecting Suspicious Activity with Falco · Slide 407

- Domain 5 — DevOps Fundamentals (10%) — Deck: divider slide 408; Slides 476–479
  - Lab 29 — Git Source Control & Branching · Slide 476
  - Lab 30 — CI/CD Pipeline with GitHub Actions (act) · Slide 477
  - Lab 31 — REST & GraphQL APIs · Slide 478
  - Lab 32 — Container Orchestration with Kubernetes (k3s) · Slide 479
- Domain 6 — Troubleshooting (12%) — Deck: divider slide 480; Slides 549–552
  - Lab 33 — Troubleshoot Deployment Issues · Slide 549
  - Lab 34 — Troubleshoot Cloud Network Issues · Slide 550
  - Lab 35 — Troubleshoot TLS, Cipher & Auth Issues · Slide 551
  - Lab 36 — Troubleshoot Container & Resource Limits · Slide 552

Resources Required

Assessment

Course Overview

The **WSQ CompTIA Certified Cloud+ Training** (CV0-004) is a 2-day, hands-on WSQ course that prepares learners for the CompTIA Cloud+ CV0-004 certification. It maps 36 practical labs to the six exam domains, all delivered on the free Killercoda Ubuntu Playground.

- **TGS Ref No:** TGS-2024049214
- **TSC:** Cloud Computing (ICT-DIT-5020-1.1)
- **Duration:** 2 days (9:00 AM – 6:00 PM each day)
- **Delivery:** Instructor-led lecture + hands-on labs on https://killercoda.com/playgrounds/scenario/ubuntu

Learning Outcomes

By the end of this course, participants will be able to:

- Explain cloud architecture, service models (IaaS/PaaS/SaaS/FaaS) and the shared responsibility model, and design highly-available, well-networked cloud solutions.
- Deploy cloud workloads using containers, virtualization, Infrastructure as Code and modern release strategies (blue-green, canary, rolling).
- Operate cloud systems with observability, logging, tracing, auto-scaling, backup/recovery and lifecycle management.
- Secure a cloud environment with vulnerability management, hardening, IAM/RBAC, secrets management, WAF and runtime threat detection.
- Apply DevOps fundamentals — source control, CI/CD, APIs and container orchestration.
- Troubleshoot common cloud deployment, network, security and container issues methodically.

Exam Domain Coverage

Domain	Exam Weight	Labs
1. Cloud Architecture	23%	1, 2, 3, 4, 5, 6, 7, 8
2. Deployment	19%	9, 10, 11, 12, 13, 14, 15
3. Operations	17%	16, 17, 18, 19, 20, 21
4. Security	19%	22, 23, 24, 25, 26, 27, 28
5. DevOps Fundamentals	10%	29, 30, 31, 32
6. Troubleshooting	12%	33, 34, 35, 36

Daily Schedule

The **Slides** column maps each session to the matching pages in the course slide deck (*CompTIA Cloud+ (CV0-004) Slides v2.0*, 561 slides).

Day 1 — Cloud Architecture, Deployment & Operations

Time	Duration	Topic / Activity	Method	Slides
9:00–9:30	30 min	Registration, AM Digital Attendance, trainer & learner introductions, Ground Rules, Course Outline, Learning Outcomes	Admin	—
9:30–10:45	1 hr 15 min	**Domain 1 — Cloud Architecture:** cloud concepts, service models & shared responsibility, high availability, VPC networking (Labs 1–3)	Lecture + Hands-on labs	Slides 135–137
10:45–10:55	10 min	Morning break	Break	—
10:55–12:30	1 hr 35 min	**Domain 1 continued:** storage tiers, microservices, containerization, virtualization, relational vs non-relational DBs (Labs 4–8)	Hands-on labs	Slides 138–142
12:30–1:15	45 min	Lunch break	Break	—
1:15–3:00	1 hr 45 min	**Domain 2 — Deployment:** public/private/hybrid models, blue-green, canary and rolling deployments (Labs 9–12)	Lecture + Hands-on labs	Slides 226–229
3:00–3:10	10 min	Afternoon break	Break	—
3:10–4:30	1 hr 20 min	**Domain 2 continued:** Infrastructure as Code (Terraform), Configuration as Code (Ansible), JSON & YAML scripting logic (Labs 13–15)	Hands-on labs	Slides 230–232
4:30–6:00	1 hr 30 min	**Domain 3 — Operations:** observability with Prometheus & Grafana, centralized logging (ELK), distributed tracing (Jaeger) (Labs 16–18)	Lecture + Hands-on labs	Slides 304–306

Day 2 — Security, DevOps, Troubleshooting & Assessment

Time	Duration	Topic / Activity	Method	Slides
9:00–9:15	15 min	AM Digital Attendance, recap of Day 1	Admin	—
9:15–10:45	1 hr 30 min	**Domain 3 continued:** horizontal auto-scaling, backup & recovery (restic), patching & lifecycle management (Labs 19–21)	Hands-on labs	Slides 307–309
10:45–10:55	10 min	Morning break	Break	—
10:55–12:30	1 hr 35 min	**Domain 4 — Security:** vulnerability scanning (Trivy), CIS hardening (Lynis), IAM & RBAC (Keycloak), bastion + MFA (Labs 22–25)	Lecture + Hands-on labs	Slides 401–404
12:30–1:15	45 min	Lunch break	Break	—
1:15–2:15	1 hr	**Domain 4 continued:** secrets management (Vault), WAF (ModSecurity + OWASP CRS), runtime detection (Falco) (Labs 26–28)	Hands-on labs	Slides 405–407
2:15–3:15	1 hr	**Domain 5 — DevOps Fundamentals:** Git source control & branching, CI/CD pipelines, REST/GraphQL APIs, Kubernetes with k3s (Labs 29–32)	Lecture + Hands-on labs	Slides 476–479
3:15–3:25	10 min	Afternoon break	Break	—
3:25–4:00	35 min	**Domain 6 — Troubleshooting:** deployment, network, security and container/resource triage (Labs 33–36); **Briefing for Assessment**	Hands-on + Briefing	Slides 549–552
4:00–6:00	2 hr	**Assessment:** Written Assessment (1 hr) + Practical Performance (1 hr). TRAQOM	Assessment	—

		survey and answer submission on the LMS.		

Topic-by-Topic Breakdown

Domain 1 — Cloud Architecture (23%) — Deck: divider slide 19; Slides 135–142

Cloud concepts, service and deployment models, the shared responsibility model, high availability, networking (VPC), storage tiers, virtualization, databases and cloud-native / microservices design.

Lab 1 — Cloud Service Models & Shared Responsibility · Slide 135

Goal: Run a local stand-in for each cloud service model and articulate who is responsible for what under the shared responsibility model.

Key concepts:

- The four service models — IaaS, PaaS, SaaS, FaaS — and what each abstracts away
- Shared responsibility: who owns data, app, runtime, OS and hardware at each layer
- FaaS traits: event-driven, scales to zero, billed per invocation
- Mapping real services (EC2, managed Postgres, M365, Lambda) to each model

Tools: Docker, curl, jq, nginx, postgres:16, nextcloud

Lab 2 — High Availability with HAProxy & Keepalived · Slide 136

Goal: Build a health-checked load balancer across two AZ backends and add VRRP failover so a virtual IP survives a load-balancer failure.

Key concepts:

- Multi-AZ load balancing and round-robin distribution across backends
- Active health checks that auto-remove unhealthy instances
- Keepalived / VRRP floating virtual IP (VIP) for LB redundancy
- Active-passive HA: the BACKUP node takes the VIP when the MASTER dies

Tools: HAProxy, Keepalived, Docker, nginx:alpine, curl

Lab 3 — Cloud Networking with VPC Namespaces · Slide 137

Goal: Construct a full VPC — two subnets, a router, a security-group rule and a NAT gateway — entirely from Linux network namespaces.

Key concepts:

- A VPC/subnet is a kernel network feature (network namespace = subnet)
- Route tables, default routes and IP forwarding for inter-subnet routing

- Security groups operate per-flow at L3/L4 (iptables = AWS SG / Azure NSG)
- NAT gateway (MASQUERADE) lets private subnets reach the internet

Tools: iproute2 (ip netns), iptables, sysctl, ping

Lab 4 — Storage Tiers (Block, Object, File) · Slide 138

Goal: Create one block, one object and one file store locally, then benchmark IOPS to explain why storage tiers (and their prices) differ.

Key concepts:

- Three storage types: block (EBS), object (S3), file (EFS/Azure Files)
- Object storage is key-addressed over HTTP with no filesystem; file is POSIX/NFS
- Measuring random-write IOPS with fio to compare tier performance
- Hot / warm / cold / archive tiering and how cost follows performance

Tools: fio, losetup/mkfs.ext4, MinIO + mc, NFS, Docker

Lab 5 — Microservices & Service Discovery · Slide 139

Goal: Deploy three loosely-coupled microservices behind Consul and demonstrate name-based service discovery, fan-out and resilience under a single failure.

Key concepts:

- Services register / deregister with a service registry (Consul)
- Clients resolve services by name (users.service.consul), not IP
- Fan-out pattern: one request triggers parallel calls to many services
- Loose coupling: killing one service leaves the others running

Tools: HashiCorp Consul, Docker bridge network, curl, jq, dig

Lab 6 — Containerization with Docker · Slide 140

Goal: Practise every core containerization sub-objective — build/run/expose, ephemeral vs persistent storage, public/private registries and a Compose preview.

Key concepts:

- Image lifecycle: build, tag, run, push (Dockerfile FROM/COPY/EXPOSE)
- Port mapping (-p host:container)
- Ephemeral storage vs persistent volumes
- Public vs private image registries (Docker Hub vs a local registry:2)

Tools: Docker, Dockerfile, registry:2, Docker Compose

Lab 7 — Virtualization with QEMU/KVM · Slide 141

Goal: Create and clone a QEMU VM to understand core virtualization concepts and contrast VMs with containers.

Key concepts:

- Hypervisor virtualization vs containerization (own kernel vs shared kernel)
- qcow2 copy-on-write disks and linked clones (basis of snapshots/templates)
- Hardware-assisted virtualization (Intel vmx / AMD svm)
- VM network types (user-mode NAT vs bridged) and host affinity / CPU pinning

Tools: QEMU, qemu-img, libvirt, bridge-utils, Alpine ISO

Lab 8 — Relational vs Non-Relational Databases · Slide 142

Goal: Run and compare a self-managed relational DB against a document DB to learn when to choose each and how managed services shift responsibility.

Key concepts:

- Relational model: strict schema, ACID, foreign keys, JOINs (Postgres)
- Document model: schemaless, nested documents, built-in sharding (MongoDB)
- Self-managed vs provider-managed responsibility split
- Workload-to-engine selection and provider-managed equivalents (RDS, Cosmos…)

Tools: Docker, PostgreSQL 16 + psql, MongoDB + mongosh

Domain 2 — Deployment (19%) — Deck: divider slide 143; Slides 226–232

Provisioning and configuring cloud resources: deployment models, zero-downtime release strategies (blue-green, canary, rolling), Infrastructure as Code, Configuration as Code and scripting logic.

Lab 9 — Public, Private & Hybrid Cloud Models · Slide 226

Goal: Simulate public (LocalStack), private (MinIO) and hybrid deployment models to compare their cost, control and elasticity trade-offs.

Key concepts:

- Public (multi-tenant, pay-per-use) vs private (single-tenant, high CapEx)
- Hybrid: the same S3 API spans both clouds and enables cloud bursting
- Community cloud is a policy/access concept, not new infrastructure
- A consistent API (S3) can span deployment models

Tools: LocalStack, MinIO + mc, AWS CLI, WireGuard, Docker

Lab 10 — Blue-Green Deployment with Nginx · Slide 227

Goal: Run blue (v1) and green (v2) side by side and switch live traffic instantly via an Nginx reload to achieve zero-downtime deployment.

Key concepts:

- Blue-green keeps two parallel environments; cutover is a config change
- Atomic traffic cutover via 'nginx -s reload' with no dropped connections

- Smoke-test the new version out of band before flipping traffic
- Instant, symmetrical rollback; trade-off is 2× resources

Tools: Nginx, Docker (nginx:alpine), curl

Lab 11 — Canary Deployment with HAProxy · Slide 228

Goal: Gradually shift live traffic from v1 to v2 using HAProxy server weights to safely validate a new release before full promotion.

Key concepts:

- Canary = a small percentage of real traffic on the new version first
- Weight-based routing (90/10 → 50/50 → 100%) = ALB weighted target groups / Istio
- Gradual promotion and instant weight-reset rollback
- Always pair canary with metrics and alerts

Tools: HAProxy, Docker (nginx:alpine), curl

Lab 12 — Rolling Deployment with Docker Compose · Slide 229

Goal: Deploy three load-balanced replicas and upgrade them one at a time to perform a zero-downtime rolling update.

Key concepts:

- Rolling = replace replicas one (or a batch) at a time while others serve
- Produces a brief mixed-version window; cheaper than blue-green
- Load balancing across replicas via HAProxy server-template + resolvers
- Always pair with a readiness check before progressing

Tools: Docker Compose, HAProxy, curl

Lab 13 — Infrastructure as Code with Terraform · Slide 230

Goal: Write a Terraform config that provisions an S3 bucket on LocalStack and experience plan/apply, state, drift detection and versioning.

Key concepts:

- IaC: declarative configuration becomes real cloud resources
- The plan-before-apply workflow
- Drift detection compares real vs desired state
- State (terraform.tfstate) is the source of truth and must be stored safely

Tools: Terraform, LocalStack, awslocal, Git, Docker

Lab 14 — Configuration as Code with Ansible · Slide 231

Goal: Write an Ansible playbook that installs and configures Nginx idempotently and prove drift correction.

Key concepts:

- Configuration as Code: YAML playbooks declare desired state
- Idempotency — tasks only act when current state differs (changed vs ok)
- Re-running restores desired state after manual tampering
- Variables, conditionals (when:) and roles compose larger configs

Tools: Ansible, ansible-galaxy, YAML, curl

Lab 15 — JSON & YAML Scripting Logic · Slide 232

Goal: Parse, transform, validate and convert JSON ↔ YAML while exercising the exam's scripting-logic primitives.

Key concepts:

- Read, write, validate and convert JSON and YAML (same data, two formats)
- Shared data types: string, number, boolean, array, object
- Scripting-logic primitives: variables, conditionals, operators, functions (jq)
- Validate at the boundary — fail early on malformed input

Tools: jq, yq, diff, web validators

Domain 3 — Operations (17%) — Deck: divider slide 233; Slides 304–309

Running cloud systems day-to-day: observability (metrics, logs, traces), alerting, auto-scaling, backup & disaster recovery, and patching / lifecycle management.

Lab 16 — Observability with Prometheus & Grafana · Slide 304

Goal: Scrape a node's metrics with Prometheus, visualise them in Grafana and trigger a CPU alert.

Key concepts:

- Metrics pillar: Prometheus scrapes a source (node-exporter) on an interval
- Grafana visualises Prometheus as a data source
- Threshold alerting: a PromQL rule fires when a condition holds for a duration
- The operational loop: alerting → triage → response

Tools: Prometheus, Grafana, node-exporter, stress-ng, Docker

Lab 17 — Centralized Logging with the ELK Stack · Slide 305

Goal: Run ELK, ship logs into it, and query and retain them through the Elasticsearch API.

Key concepts:

- The centralized logging pipeline: collect → ship → store → query
- Elasticsearch (store/index), Logstash (ingest/transform), Kibana (query/visualise)

- Retention enforced with ILM (Index Lifecycle Management) policies
- ELK and OpenSearch share the same query DSL

Tools: Elasticsearch, Logstash, Kibana, Docker, curl, jq

Lab 18 — Distributed Tracing with Jaeger · Slide 306

Goal: Run Jaeger, instrument a Python 'checkout' path with OpenTelemetry and view a distributed trace of parent/child spans.

Key concepts:

- Tracing is the third pillar of observability (metrics/logs/traces)
- Spans are the unit; a trace is a set of causally-linked spans across services
- OpenTelemetry is the vendor-neutral instrumentation API (OTLP export)
- A trace reveals the critical path — the slowest span — of one request

Tools: Jaeger, OpenTelemetry Python SDK, Docker, python3, curl

Lab 19 — Horizontal Auto-Scaling Simulation · Slide 307

Goal: Simulate triggered, scheduled and manual horizontal scaling using a Bash controller that watches CPU and adds/removes Docker replicas.

Key concepts:

- Three scaling trigger families: triggered/load-based, scheduled, manual
- An auto-scaler is a control loop that watches a metric and adjusts replicas
- Horizontal scaling (add replicas) vs vertical scaling (resize one instance)
- Horizontal scaling is the cloud-native default (AWS ASG, K8s HPA)

Tools: Docker Compose, stress-ng, cron, nginx:alpine

Lab 20 — Backup & Recovery with restic · Slide 308

Goal: Perform encrypted full/incremental/differential backups to off-site S3 storage, then test recoverability and verify integrity.

Key concepts:

- Full vs incremental vs differential backup semantics and restore trade-offs
- Encrypted, deduplicating backups to off-site S3-compatible storage
- Recoverability testing (granular and bulk restore) is mandatory
- Integrity verification and retention policies (keep-daily/weekly + prune)

Tools: restic, MinIO, Docker

Lab 21 — Patching & Lifecycle Management · Slide 309

Goal: Simulate a cloud resource lifecycle from provision through minor patch, major upgrade, testing and clean decommissioning.

Key concepts:

- The full lifecycle: provision → configure → monitor → patch → scale → backup → decommission
- Minor patches (no behaviour change) vs major upgrades (breaking) risk profiles
- Persistent data must survive upgrades; ephemeral state may vanish
- EoL/EoS items must be replaced, not patched; decommission = archive + revoke + delete

Tools: apt, unattended-upgrades, Docker, curl

Domain 4 — Security (19%) — Deck: divider slide 310; Slides 401–407

Securing the cloud: vulnerability management, CIS hardening, IAM & RBAC, secure access (bastion + MFA), secrets management, web application firewalls and runtime threat detection.

Lab 22 — Vulnerability Scanning with Trivy · Slide 401

Goal: Scan a container image, filesystem and IaC config with Trivy following the scope → identify → assess → remediate workflow.

Key concepts:

- The four-step vulnerability-management workflow: scope → identify → assess → remediate
- CVE (identifier) + CVSS (severity) as the universal vulnerability vocabulary
- Three scan surfaces: container image, filesystem (secrets), IaC config
- The CI 'gate' — a non-zero exit code fails the build

Tools: Trivy, Docker, sample Terraform

Lab 23 — CIS Benchmark Audit with Lynis · Slide 402

Goal: Audit the VM against the CIS Ubuntu Benchmark with Lynis, score it, remediate findings and re-audit to confirm improvement.

Key concepts:

- Benchmarks (CIS) are objective, measurable targets; hardening closes the gap
- The Lynis hardening index is a quantifiable score
- Findings map to specific CIS controls (file permissions, kernel modules)
- Different benchmarks per layer: OS, Docker Bench, kube-bench, Prowler

Tools: Lynis, chmod, sysctl, Docker Bench Security

Lab 24 — IAM & RBAC with Keycloak · Slide 403

Goal: Stand up Keycloak and build a full IAM stack — realm, roles, users, an OAuth2 client — then authenticate and inspect RBAC roles in the JWT.

Key concepts:

- IAM building blocks: identity provider, realms (tenants), users, roles, clients
- OAuth 2.0 + OIDC token flow (bearer JWT access token)
- RBAC — roles propagate via JWT realm_access.roles claims
- Authentication models: local, federation/SAML, OIDC, LDAP, MFA

Tools: Keycloak, Docker, curl, jq

Lab 25 — Bastion Host with SSH Key + MFA · Slide 404

Goal: Configure key-based SSH with TOTP MFA and route access through a locked-down bastion jump host — the canonical secure cloud-VM access pattern.

Key concepts:

- SSH public-key authentication replacing passwords
- TOTP multi-factor auth chained via AuthenticationMethods / PAM
- Bastion / jump-host pattern (internet → bastion → private subnet)
- Restricting bastion access to corporate CIDRs; audit trails for compliance

Tools: OpenSSH, ssh-keygen, Google Authenticator (PAM), UFW, journalctl

Lab 26 — Secrets Management with HashiCorp Vault · Slide 405

Goal: Run Vault to store static secrets, mint auto-expiring dynamic DB credentials and audit every access.

Key concepts:

- Never hard-code secrets; reference them from a secrets manager (Vault KV v2)
- Secret versioning for audit and rollback
- Dynamic, short-lived DB credentials with lease/TTL shrink the blast radius
- Audit logging of every secret access; cloud equivalents (Secrets Manager, Key Vault)

Tools: HashiCorp Vault, Docker, PostgreSQL, curl, jq

Lab 27 — Web Application Firewall with ModSecurity · Slide 406

Goal: Deploy Nginx + ModSecurity + the OWASP Core Rule Set and watch it block SQLi, XSS and path-traversal, then tune false positives.

Key concepts:

- WAFs inspect HTTP semantics (query strings, bodies, headers) — not just IP/port
- OWASP CRS catches the OWASP Top 10 out of the box (SQLi, XSS, LFI)
- Anomaly scoring with a block threshold and paranoia levels
- False-positive tuning done test → staging → prod; ACL vs WAF vs Security Group

Tools: Docker (owasp/modsecurity-crs), Nginx, OWASP CRS, curl

Lab 28 — Detecting Suspicious Activity with Falco · Slide 407

Goal: Run Falco and trigger runtime detections that map to CV0-004 attack categories, then route alerts to Prometheus / SIEM.

Key concepts:

- Runtime security uses kernel signals (eBPF) to spot bad behaviour
- Common detections: shell-in-container, sensitive-file reads, odd outbound connections
- Detections map to CV0-004 attack categories (privesc, cryptojacking, metadata abuse)
- Baseline-deviation alerting; events flow into Prometheus / SIEM

Tools: Falco (eBPF), Docker, falcosidekick, journalctl

Domain 5 — DevOps Fundamentals (10%) — Deck: divider slide 408; Slides 476–479

Source control and branching, CI/CD pipelines, web-service APIs (REST / GraphQL) and container orchestration with Kubernetes.

Lab 29 — Git Source Control & Branching · Slide 476

Goal: Exercise every CV0-004 source-control verb — commit, push, branch, merge, PR review, conflict resolution — against a self-hosted Gitea server.

Key concepts:

- Core Git workflow: commit → push → branch → PR → merge
- Feature branches and pull requests (the code-review sub-objective)
- Merge-conflict resolution as a normal part of the workflow
- Branch protection: require PR + approval, require CI green, block force-push

Tools: git, Gitea (self-hosted), curl, jq

Lab 30 — CI/CD Pipeline with GitHub Actions (act) · Slide 477

Goal: Build and run a local test → build → scan → deploy CI/CD pipeline with act, mapping each stage to CV0-004 CI/CD concepts.

Key concepts:

- A pipeline is YAML + jobs + dependencies (needs:) forming a DAG
- The canonical flow: test → build → scan → deploy
- Artifacts and packaging (container image + Dockerfile)
- Running Actions locally with act to iterate without pushing

Tools: act (nektos/act), Docker, git, pytest, Trivy

Lab 31 — REST & GraphQL APIs · Slide 478

Goal: Call a public REST API, host a local GraphQL server, stream over WebSockets and preview pub/sub event-driven messaging.

Key concepts:

- REST is stateless, uses HTTP verbs, one resource per URL, described by OpenAPI
- GraphQL: a single endpoint where the client picks exactly which fields to return
- Comparison of styles: REST vs SOAP vs GraphQL vs gRPC
- WebSockets for real-time streams; pub/sub as the base of event-driven architecture

Tools: curl, jq, GraphQL Yoga, websocat, redis-cli

Lab 32 — Container Orchestration with Kubernetes (k3s) · Slide 479

Goal: Install k3s and deploy, expose, scale, roll out/rollback, add persistence, self-healing and RBAC on a real Kubernetes cluster.

Key concepts:

- k3s is a minimal single-binary Kubernetes cluster installable in one command
- Deployments manage replica state; Services (NodePort) expose pods
- Horizontal scaling, rolling updates, rollout history and rollback
- Self-healing via liveness probes; PVCs for persistence; RBAC for authorization

Tools: k3s, kubectl, YAML manifests, curl

Domain 6 — Troubleshooting (12%) — Deck: divider slide 480; Slides 549–552

Methodically diagnosing and fixing deployment, network, security and container / resource issues using a repeatable triage workflow.

Lab 33 — Troubleshoot Deployment Issues · Slide 549

Goal: Reproduce and fix each CV0-004 6.1 deployment failure mode locally — resources, permissions, oversubscription, sizing, deprecation, quotas, regions.

Key concepts:

- Resource allocation / sizing failures cause OOM kills — size to peak load
- Permission misconfiguration fixed by mounting writable tmpfs/volumes
- Outdated/EoL definitions and deprecation are deployment risks — pin supported tags
- API throttling returns HTTP 429 — fix with quota increase, backoff, batching

Tools: Docker, curl, jq, region matrices

Lab 34 — Troubleshoot Cloud Network Issues · Slide 550

Goal: Reproduce and resolve CV0-004 6.2 network failures using an L2→L7 diagnostic ladder (DHCP, DNS, NTP, NAT, HTTP codes, routing, IP overlap).

Key concepts:

- DNS and NTP break 'everything else' when they fail
- NAT (MASQUERADE + ip_forward) enables outbound from private subnets
- HTTP status codes localise the failure layer (4xx client, 5xx server, 429 rate-limit)
- IP overlap needs NAT/re-IP; DHCP scope exhaustion needs a bigger subnet

Tools: dig/nslookup, ip, iptables, mtr, chrony, tcpdump

Lab 35 — Troubleshoot TLS, Cipher & Auth Issues · Slide 551

Goal: Reproduce and fix CV0-004 6.3 security failures from the CLI — deprecated ciphers, privesc, unauthorized access, leaked credentials, vulnerabilities.

Key concepts:

- Disable deprecated TLS (SSLv3/TLS 1.0/1.1); require TLS 1.2+ with AEAD ciphers
- Detect privilege escalation via auditd file watches and SUID hunting
- Detect brute force in SSH logs (fail2ban); find leaked credentials with gitleaks
- TLS handshake errors map to specific root causes (missing CA, cipher mismatch, expiry)

Tools: openssl, nmap, auditd, fail2ban, gitleaks, Trivy

Lab 36 — Troubleshoot Container & Resource Limits · Slide 552

Goal: Combine the Lab 33–35 methods to triage real container failures with a repeatable state → logs → stats → mounts → network workflow.

Key concepts:

- CrashLoopBackOff = process exits immediately; fix the entrypoint so it stays up
- OOMKilled is the smoking gun (OOMKilled=true); raise the limit or fix the leak
- Image-pull failures come from wrong names or missing registry auth
- The daily five: OOM, CrashLoop, ImagePull, DNS, read-only-fs; triage in a fixed order

Tools: docker (inspect/stats/logs), a triage.sh script, k9s/ctop

Resources Required

- A laptop / PC with a modern web browser and internet access.
- The free Killercoda Ubuntu Playground: https://killercoda.com/playgrounds/scenario/ubuntu
- Course repository (labs source): https://github.com/tertiarycourses/TGS-2024049214-CompTIA-Certified-Cloud-Training

- Learner Guide and slide deck (download from the LMS: https://lms-tms.tertiaryinfotech.com/).
- No local installation, cloud account or credit card is required.

Assessment

Assessment is held in the final **2 hours (4:00–6:00 PM on Day 2)** and comprises:

- **Written Assessment (WA)** — 1 hour, open-ended short-answer knowledge questions drawn from the course slides / modules.
- **Practical Performance (PP)** — 1 hour, activity-based tasks the learner performed in the labs (mapped to the learning outcomes).

Assessment flow: (1) TRAQOM — scan the QR code on the LMS and complete the survey; (2) Assessment Digital Attendance; (3) Assessment (WA + PP); (4) Submit answers on the LMS; (5) Sign the Assessment Summary Record.

Courseware and the assessment are on the LMS: https://lms-tms.tertiaryinfotech.com/

Criteria for funding: learners must attend at least 75% of the course and be assessed as **Competent (C)** in both the Written Assessment and the Practical Performance.